JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ForkJoinWorkerThread

java.lang.Object
  java.lang.Thread
    java.util.concurrent.ForkJoinWorkerThread

All Implemented Interfaces:

Runnable

```
public class ForkJoinWorkerThread
extends Thread
```

A thread managed by a ForkJoinPool, which executes ForkJoinTasks. This class is subclassable solely for the sake of adding functionality -- there are no overridable methods dealing with scheduling or execution. However, you can override initialization and termination methods surrounding the main task processing loop. If you do create such a subclass, you will also need to supply a custom ForkJoinPool.ForkJoinWorkerThreadFactory to use it in a ForkJoinPool.

Since:

1.7

Nested Class Summary

Nested classes/interfaces declared in class Thread
Thread.Builder, Thread.State, Thread.UncaughtExceptionHandler

Field Summary

Fields declared in class Thread
MAX_PRIORITY, MIN_PRIORITY, NORM_PRIORITY

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	**ForkJoinWorkerThread**(ThreadGroup group, ForkJoinPool pool, boolean preserveThreadLo	Creates a ForkJoinWorkerThread operating in the given thread group and pool, and with the given policy

		for preserving ThreadLocals.
protected	**ForkJoinWorkerThread**(ForkJoinPool pool)	Creates a ForkJoinWorkerThread operating in the given pool.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
ForkJoinPool	**getPool**()	Returns the pool hosting this thread.
int	**getPoolIndex**()	Returns the unique index number of this thread in its pool.
int	**getQueuedTaskCount**()	Returns a (non-negative) estimate of the number of tasks in the thread's queue.
protected void	**onStart**()	Initializes internal state after construction but before processing any tasks.
protected void	**onTermination**(Throwable exception)	Performs cleanup associated with termination of this worker thread.
void	**run**()	This method is required to be public, but should never be called explicitly.

Methods declared in class Thread

activeCount, checkAccess, clone, currentThread, dumpStack, enumerate, getAllStackTraces, getContextClassLoader, getDefaultUncaughtExceptionHandler, getId, getName, getPriority, getStackTrace, getState, getThreadGroup, getUncaughtExceptionHandler, holdsLock, interrupt, interrupted, isAlive, isDaemon, isInterrupted, isVirtual, join, join, join, join, ofPlatform, ofVirtual, onSpinWait, setContextClassLoader, setDaemon, setDefaultUncaughtExceptionHandler, setName, setPriority, setUncaughtExceptionHandler, sleep, sleep, sleep, start, startVirtualThread, threadId, toString, yield

Methods declared in class Object

equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Details

ForkJoinWorkerThread

```
protected ForkJoinWorkerThread(ThreadGroup group,
                               ForkJoinPool pool,
                               boolean preserveThreadLocals)
```

Creates a ForkJoinWorkerThread operating in the given thread group and pool, and with the given policy for preserving ThreadLocals.

Parameters:

`group` - if non-null, the thread group for this thread. Otherwise, the thread group is set to the current thread's thread group.

`pool` - the pool this thread works in

`preserveThreadLocals` - if true, always preserve the values of ThreadLocal variables across tasks; otherwise they may be cleared.

Throws:

`NullPointerException` - if pool is null

Since:

19

ForkJoinWorkerThread

```
protected ForkJoinWorkerThread(ForkJoinPool pool)
```

Creates a ForkJoinWorkerThread operating in the given pool.

Parameters:

`pool` - the pool this thread works in

Throws:

`NullPointerException` - if pool is null

Method Details

getPool

```
public ForkJoinPool getPool()
```

Returns the pool hosting this thread.

Returns:

the pool

getPoolIndex

```
public int getPoolIndex()
```

Returns the unique index number of this thread in its pool. The returned value ranges from zero to the maximum number of threads (minus one) that may exist in the pool, and does not change during the lifetime of the thread. This method may be useful for applications that track status or collect results per-worker-thread rather than per-task.

Returns:

the index number

getQueuedTaskCount

```
public int getQueuedTaskCount()
```

Returns a (non-negative) estimate of the number of tasks in the thread's queue.

Returns:

a (non-negative) estimate of the number of tasks in the thread's queue

Since:

20

See Also:

`ForkJoinPool.getQueuedTaskCount()`

onStart

```
protected void onStart()
```

Initializes internal state after construction but before processing any tasks. If you override this method, you must invoke `super.onStart()` at the beginning of the method. Initialization requires care: Most fields must have legal default values, to ensure that attempted accesses from other threads work correctly even before this thread starts processing tasks.

onTermination

```
protected void onTermination(Throwable exception)
```

Performs cleanup associated with termination of this worker thread. If you override this method, you must invoke `super.onTermination` at the end of the overridden method.

Parameters:

`exception` - the exception causing this thread to abort due to an unrecoverable error, or `null` if completed normally

run

```
public void run()
```

This method is required to be public, but should never be called explicitly. It performs the main run loop to execute `ForkJoinTask`s.

Specified by:

`run` in interface `Runnable`

Overrides:

`run` in class `Thread`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.